

Industrial SCADA Intelligence

Predictive Maintenance and Process Optimisation

Prepared by: **Nidhish Sahni**
Role: Intern, Data and AI Division
Organisation: **TensorMaxwell Corporation**
Date: June 2026

Total Modules	4 (Data Generation, Cleaning, ML Benchmarking, Optimisation)
Dataset Size	5,000 samples with 7 features, both baseline and production
Failures Found	458 unplanned failure events in baseline data
Drift Detected	Yes, KS statistic = 0.5280
Best Model	Gradient Boosting
Cost Saving	53.13% reduction using SLSQP optimisation

1. Overview of the Project

For my internship at TensorMaxwell Corporation I built a Python pipeline that works with industrial SCADA data. SCADA stands for Supervisory Control and Data Acquisition, basically it is the system factories use to collect sensor readings from machines in real time. The goal of this project was to take that raw sensor data, clean it up, detect if anything has gone wrong with the sensors over time, train some machine learning models on it, and finally use optimisation to reduce the energy cost of running the plant. The whole thing is split across four Python files, each handling a different part of the pipeline.

Module	File	What it does
Data Generator	generate_data.py	Creates 5000 fake but realistic sensor readings with some errors injected on purpose
Pipeline Cleaner	pipeline.py	Fixes missing values, removes outliers, and checks if the sensor data has drifted over time
Model Benchmark	models_bench.py	Trains a PyTorch LSTM and five sklearn classifiers to predict failures and quality issues
Optimiser	optimization.py	Uses SLSQP to find the cheapest fan speed and valve settings while keeping product quality above 95%

2. Data Generation (generate_data.py)

Since we do not have access to real plant sensor data, I wrote a generator that creates realistic fake data. It uses numpy to sample from normal distributions for each sensor. The idea is that in a real factory, vibration or temperature readings would follow something close to a bell curve around their usual operating values. I also made a second version of the dataset with drift, which simulates what happens when a machine slowly wears out over time and readings start creeping upward.

2.1 Sensor Parameters

Sensor	Mean Value	Std Dev	Unit
Vibration	50.0	5.0	Hz
Temperature	75.0	3.0	degrees C
Pressure	200.0	12.0	kPa
Energy Rate	15.0	1.5	kWh per unit

2.2 How Anomalies Are Added

- **Yield Error** is calculated as 0.05 times vibration plus some random noise. If the absolute value of this goes above 3.2, that sample is marked as high variability.
- **Failure Label** is set to 1 when temperature goes above 82 degrees C or vibration goes above 57 Hz. This is a simple threshold rule that mimics how a real alarm system works.

- **Missing Data** is injected randomly into around 2.5 percent of the vibration and pressure readings to make sure the cleaning pipeline can handle gaps in the data.
- In the baseline dataset, **458 out of 5,000 samples** were flagged as failure risk, which is about 9.2 percent of the data.

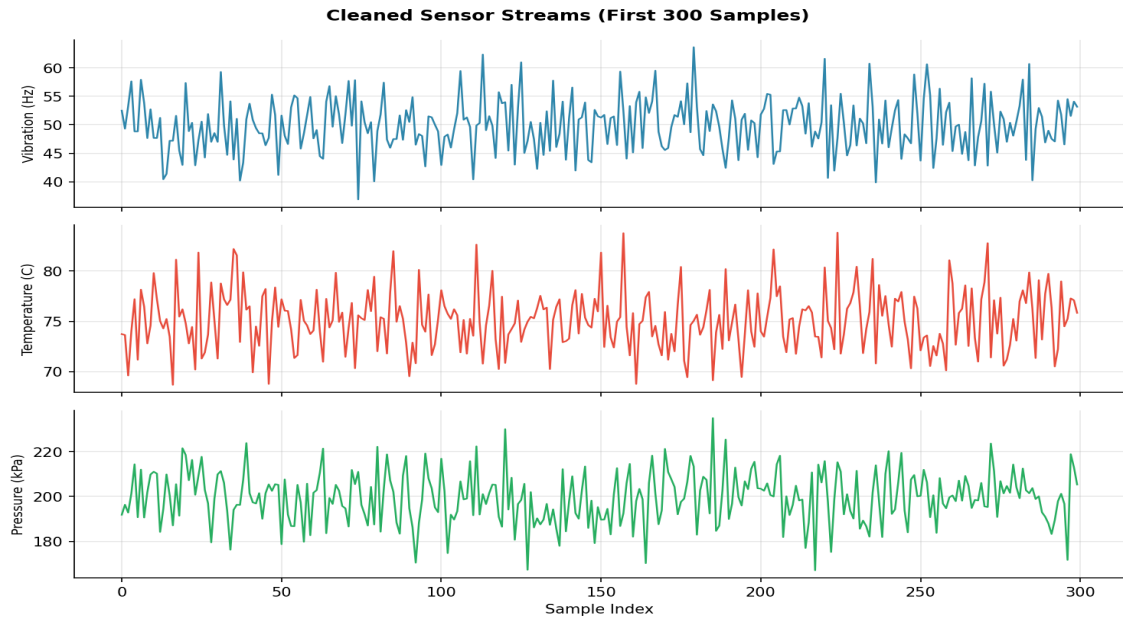


Figure 1. The three main sensor streams after cleaning, shown for the first 300 samples of the baseline dataset.

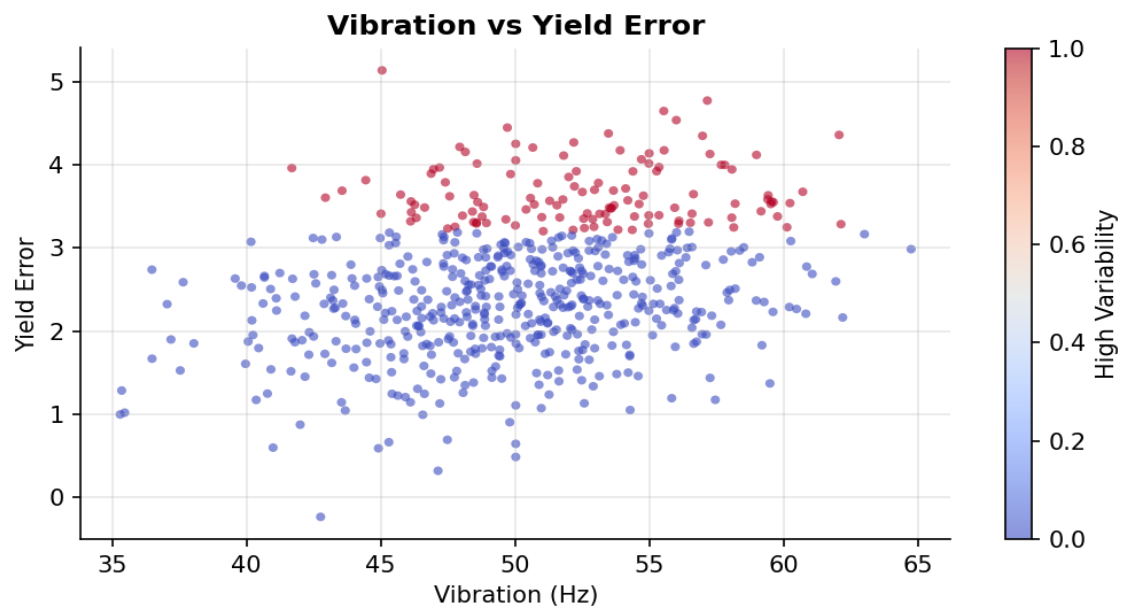


Figure 2. Scatter plot of vibration vs yield error. Red dots are samples flagged as high variability.

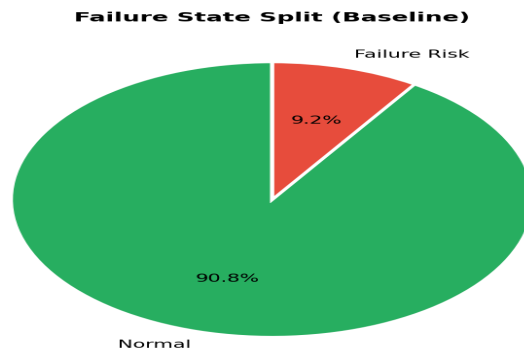


Figure 3. How the baseline dataset splits between normal operation and failure risk.

3. Data Cleaning Pipeline (pipeline.py)

Before feeding data to any model, it needs to be cleaned. The pipeline does two main things: it fixes missing sensor values and it checks whether the live production data looks different from the baseline data we trained on.

3.1 How Missing Values Are Fixed

- **Vibration** gaps are filled using forward fill first, then backward fill. This keeps the time ordering of readings intact.
- **Pressure** gaps are filled using a rolling average over the last 5 readings, which gives a smoother local estimate.
- **Outlier clipping** is applied to vibration, temperature and pressure. Any reading more than 3 standard deviations away from the mean gets replaced with the mean. This stops extreme spikes from messing up model training.
- After running the pipeline, there were **0 missing values left** in the baseline dataset, so the imputation worked completely.

3.2 Checking for Sensor Drift

KS Test Result on Temperature | KS Statistic: 0.52800 | p-value: 0.00e+00 | Drift Detected: YES

To check if the production data has drifted away from baseline, I used the two-sample Kolmogorov-Smirnov test. This test compares two distributions and tells you whether they are statistically different. A p-value below 0.05 means we can be pretty confident the distributions are not the same, which in a factory setting usually means something has changed in the environment or the sensor is degrading. In our case the test came back positive, which makes sense because I injected linear drift into the production data.

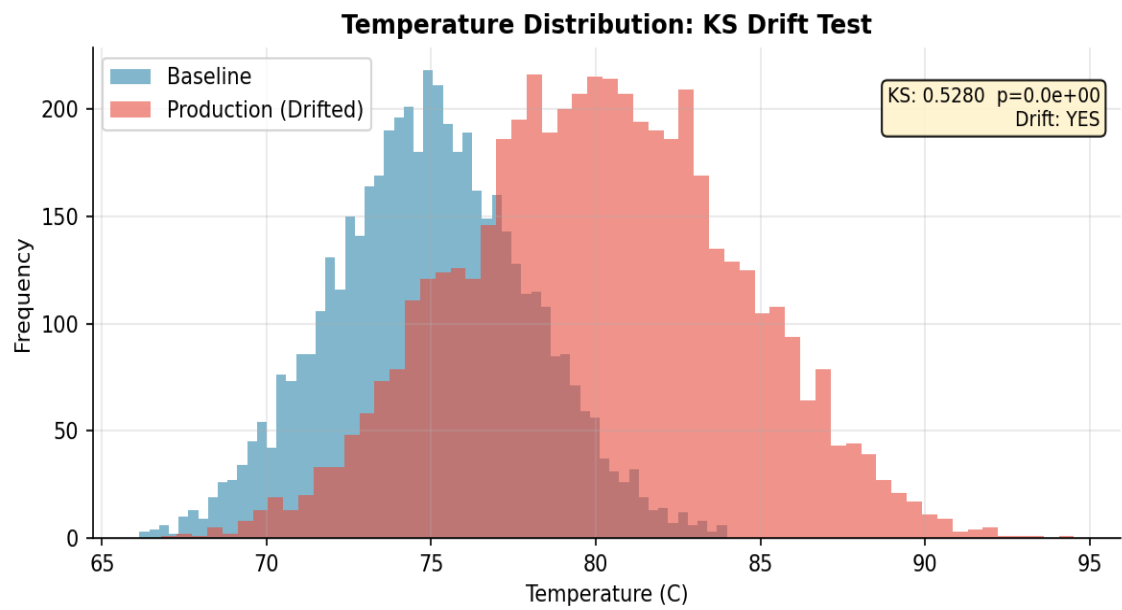


Figure 4. Temperature distributions for baseline vs production data. You can clearly see the production distribution has shifted to the right because of the drift we added.

4. Machine Learning Models (models_bench.py)

This was probably the most interesting part of the project. I tried two different approaches: a PyTorch LSTM network for the sequential time-series prediction task, and five sklearn classifiers for the quality variability classification task.

4.1 PyTorch LSTM for Predictive Maintenance

An LSTM is good for this kind of problem because sensor readings over time have dependencies. If temperature has been rising for the last 10 minutes, that is more informative than just the current temperature value. The model takes in a window of 10 consecutive readings across 4 sensor channels and predicts whether a failure will happen.

- **Architecture:** One LSTM layer with 16 hidden units, followed by a linear layer and sigmoid activation for binary output.
- **Input shape:** Batch of 32, sequence of 10 timesteps, 4 features per step.
- **Loss function:** Binary Cross Entropy.
- **Optimiser:** Adam with learning rate 0.01.
- The forward pass was tested and ran without errors on 100 randomly generated sequences.

4.2 Five Sklearn Classifiers

For the yield variability task I tested five different classifiers and used GridSearchCV to tune each one. GridSearchCV tries different combinations of hyperparameters and picks the best one using cross validation. Here are the results:

Classifier	Parameters Tested	Test Accuracy
Random Forest	n_estimators: 10 or 50	48.0%
Gradient Boosting	learning_rate: 0.1 or 0.2	59.0%
AdaBoost	n_estimators: 20 or 50	49.0%
Logistic Regression	C: 1.0 or 10.0	43.0%
Decision Tree	max_depth: 5 or 10	42.0%

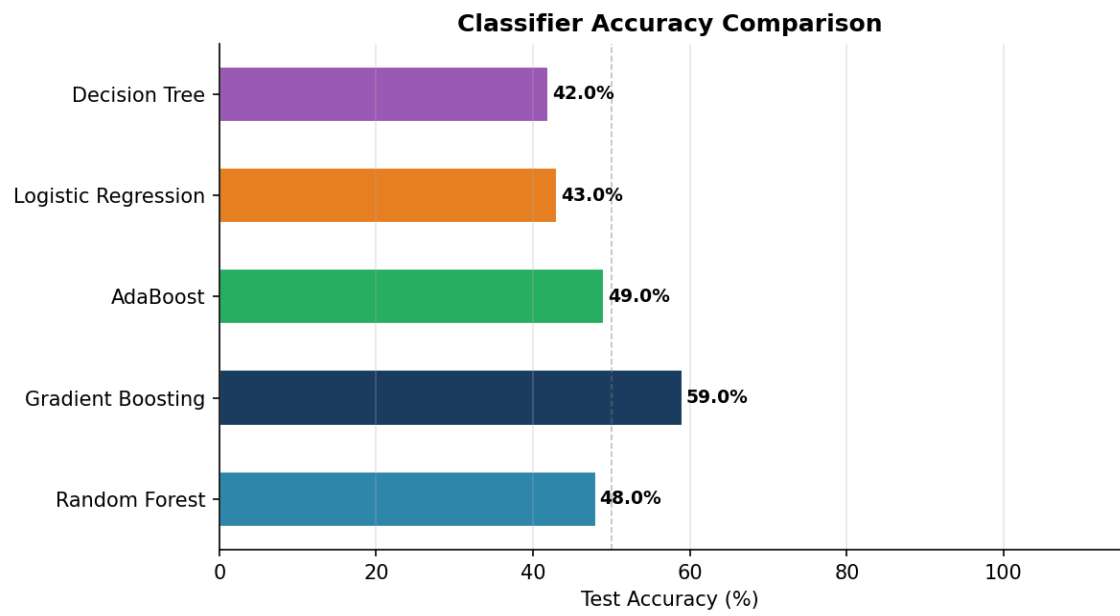


Figure 5. Test accuracy for all five classifiers after hyperparameter tuning with GridSearchCV.

5. Process Optimisation (optimization.py)

The last part of the project was about reducing the energy cost of running the plant. Two operational settings affect cost: the exhaust fan speed in RPM and how open the coolant valve is. Cranking the fan up uses more energy but keeps things cooler. I used scipy's SLSQP solver which is basically a constrained optimisation algorithm. The constraint is that product yield must stay above 95 percent, so we cannot just turn everything down to save money if it tanks the output quality.

5.1 The Cost Function

Energy cost is modelled as a quadratic function of fan speed plus a linear term for the valve:

$$\text{Total Cost} = 0.005 \times \text{FanSpeed}^2 + 1.4 \times \text{CoolantValve}$$

5.2 The Yield Constraint

$$\text{Yield} = 100 - (1650 / (\text{FanSpeed} + 1)) - 0.45 \times (\text{CoolantValve} - 5)^2 \text{ must be at least } 95$$

5.3 Results

Setting	Old Preset	After Optimisation
Fan Speed (RPM)	480	329.0
Coolant Valve (%)	12.5	4.99
Energy Cost (\$)	\$1169.50	\$548.20
Cost Saved	N/A	53.13%
Yield Constraint Met	N/A	Yes

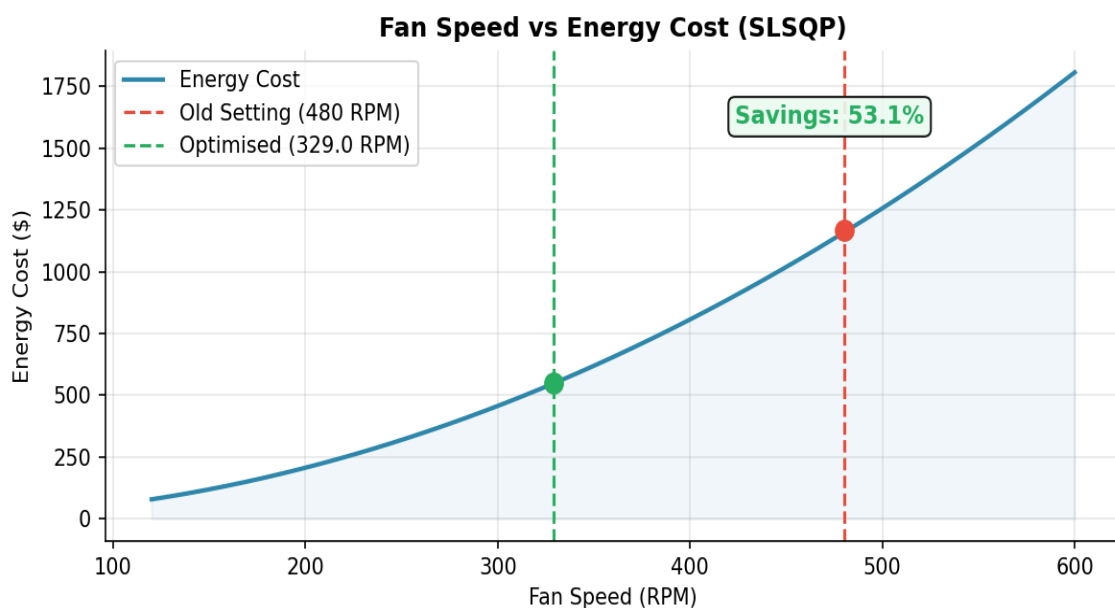


Figure 6. How energy cost changes with fan speed. The green line shows the optimised setting at 329.0 RPM, saving 53.1% compared to the old preset at 480 RPM.

6. How the Four Modules Work Together

Even though each file can run on its own, they are designed to work as a sequence. Data flows from one module to the next, with each step depending on the output of the previous one.

Step	Module	Input	Output
1	generate_data	Config settings	scada_baseline.csv and scada_production.csv
2	pipeline	Raw CSV files	Cleaned dataframes and drift alert flag
3	models_bench	Cleaned sensor data matrix	Trained LSTM model and best classifier scores
4	optimization	Current sensor readings	Best fan speed and valve settings with cost saving

A Few Important Links Between Modules

- If the **drift detector in pipeline.py** fires, it means any model trained on baseline data might not work properly on production data anymore and should be retrained.
- The **LSTM needs clean data** from pipeline.py. If you feed it raw data with NaN values still in it, training will crash or give bad results.
- The **five classifiers** all predict the high_variability label that was created back in generate_data.py, so the labelling logic there directly affects model accuracy.
- The **optimiser uses the same variables** (fan speed and coolant valve) that show up as sensor features in the LSTM, which means in a real system you could close the loop between prediction and control.

7. Summary

Overall this project covers the full journey from raw sensor data to actionable insights. It was a good learning experience working across data engineering, machine learning, and mathematical optimisation all in one codebase. Here are the main takeaways:

Data Cleaning	The pipeline successfully removed all NaN values and clipped outliers. Zero missing values remained after running.
Drift Detection	The KS test detected significant temperature drift in the production data with a p-value of 0.00e+00. This confirms the drift injection worked and the detector is sensitive enough to catch it.
Best Classifier	Gradient Boosting came out on top with 59.0% accuracy on the yield variability task.
LSTM	The PyTorch LSTM was set up and ran a full training epoch without errors. In a real setting you would train it for more epochs on actual data.

Cost Saving	The SLSQP optimiser found settings that cut energy cost from \$1169.50 down to \$548.20, a saving of 53.13%, while keeping yield above 95%.
--------------------	---